

El problema del overfitting

Source: Deep Learning with Python, by Nikhil Ketkar, Ed. Apress.



Preparado por Prof. M.A.Valle
mauricio.valle.b@uai.cl

El problema del overfitting

Si observamos la evolución de la loss función (cross-entropy para el caso de un problema de clasificación) veremos que para el set de datos de entrenamiento, el loss va disminuyendo, lo cual es esperado.

No obstante, si vemos el loss del set de datos de validación, podríamos ver que a un cierto punto comienza a aumentar en vez de disminuir.

Esto es un síntoma de overfitting

El problema del overfitting

Si observamos la evolución de la loss función (cross-entropy para el caso de un problema de clasificación) veremos que para el set de datos de entrenamiento, el loss va disminuyendo, lo cual es esperado.

No obstante, si vemos el loss del set de datos de validación, podríamos ver que a un cierto punto comienza a aumentar en vez de disminuir.

Esto es un síntoma de overfitting

El overfitting es un fenómeno por el cual el aprendizaje de los pesos de la red neuronal está ajustado en exceso a los datos de entrenamiento, reduciendo la capacidad del sistema de poder reconocer correctamente otras instancias que no son parte del set de datos de entrenamiento.

En otras palabras, el overfitting reduce la capacidad de **generalización** del sistema.

El problema del overfitting

Dos posibles soluciones a este problema:

Primero:

Obtener más data de entrenamiento para aumentar la capacidad de generalización del sistema.
No obstante esto puede no ser posible.

El problema del overfitting

Dos posibles soluciones a este problema:

Primero:

Obtener más data de entrenamiento para aumentar la capacidad de generalización del sistema.
No obstante esto puede no ser posible.

Segundo:

Modular / reducir la cantidad de información que el modelo (la NN) es permitida almacenar, es decir, limitar el aprendizaje.

Esto se hace a través de la imposición de restricciones, como por ejemplo limitar la cantidad de memorización de patrones.

Esto se denomina **regularización**.

Regularización

La manera más simple de regularización es reducir el número de parámetros para aprender, es decir, el número de pesos, es decir, por ejemplo, **disminuir la cantidad de capas escondidas (reducir la “capacidad” del modelo)**.

Regularización

La manera más simple de regularización es reducir el número de parámetros para aprender, es decir, el número de pesos, es decir, por ejemplo, **diminuir la cantidad de capas escondidas (reducir la “capacidad” del modelo)**.

Remember:

Deep-learning models tend to be good at fitting to the training data, but the real challenge is generalization, not fitting.

Regularización – reduce net.

La manera más simple de regularización es reducir el número de parámetros para aprender, es decir, el número de pesos, es decir, por ejemplo, **diminuir la cantidad de capas escondidas (reducir la “capacidad” del modelo)**.

Remember:

Deep-learning models tend to be good at fitting to the training data, but the real challenge is generalization, not fitting.

Por otro lado, no queremos underfitting, es decir, simplificar tanto el modelo que el loss sea demasiado grande, degradando la capacidad de hacer un fit.

Hay un compromiso entre *too much capacity* y *not enough capacity*.

Regularización – reduce net.

¿cómo decidimos?

No hay una fórmula. Es un asunto de materia empírica para cada problema en particular → probar / testear.

Significa probar distintas arquitecturas (no cambiar los datos!!).

Regularización – reduce net.

¿cómo decidimos?

No hay una fórmula. Es un asunto de materia empírica para cada problema en particular → probar / testear.

Significa probar distintas arquitecturas (no cambiar los datos!!).

Regularización – reduce net.

¿cómo se hace en Keras?

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Red original:

10000 vector input

1era Capa escondida 16 nodos.

2da Capa escondida 16 nodos.

Neurona de salida con sigmoid.

Regularización – reduce net.

¿cómo se hace en Keras?

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Red original:

10000 vector input

1era Capa escondida 16 nodos.

2da Capa escondida 16 nodos.

Neurona de salida con sigmoid.

```
model = models.Sequential()
model.add(layers.Dense(4, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(4, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Red reducida:

10000 vector input

1era Capa escondida 4 nodos.

2da Capa escondida 4 nodos.

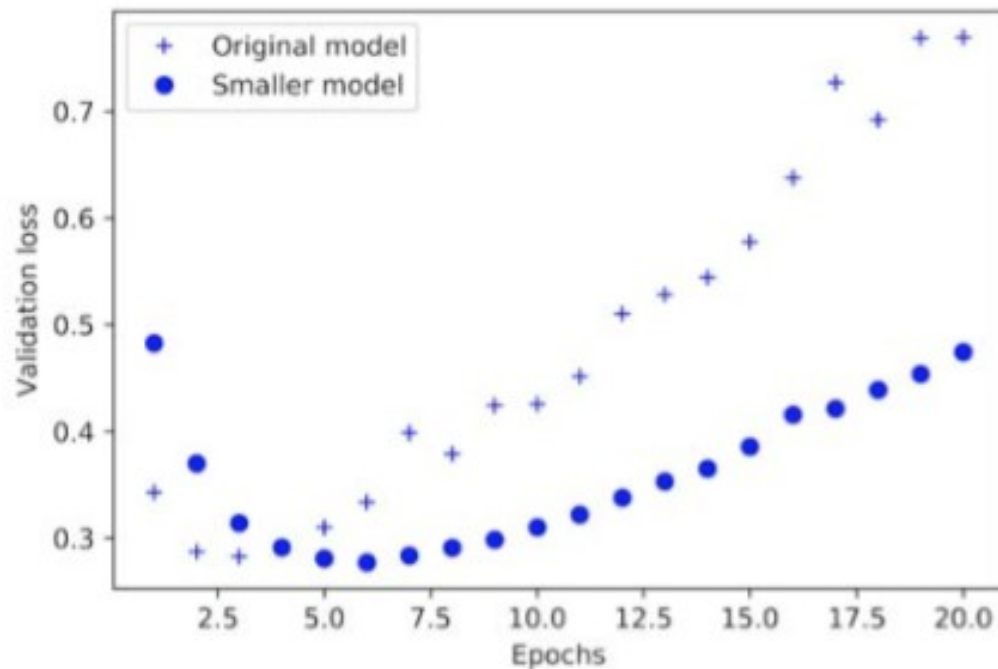
Neurona de salida con sigmoid.

Regularización – reduce net.

Validation loss de ambas redes.

Fijarse que el loss de la red más pequeña es mejor que el de la red más grande (con mayor capacidad). Se puede ver que la red más pequeña comienza a hacer overfitting más tarde que la red más grande y a una tasa menor.

Como podemos ver, no siempre se trata de una arquitectura demasiado grande!



Regularización – weight regularization.

Aquí la premisa es:

“dado un set de datos de entrenamiento y distintas arquitecturas de red, la más simple es la que tiene menor chance de overfitting”

Regularización – weight regularization.

Aquí la premisa es:

“dado un set de datos de entrenamiento y distintas arquitecturas de red, la más simple es la que es más probables que genere el menor overfitting”

Más simple significa un modelo con menor cantidad de parámetros que estimar (pesos) y cuya entropía sea la menor posible.

Para lograr esto último, ponemos a la función de pérdida un término que restrinja los valores de los pesos a valores más pequeños.

Esto se denomina **weight regularization. Equivale a agregar un término de costo a la función de pérdida por tener pesos cada vez más grandes.**

Regularización – weight regularization.

Aquí la premisa es:

“dado un set de datos de entrenamiento y distintas arquitecturas de red, la más simple es la que es más probable que genere el menor overfitting”

Más simple significa un modelo con menor cantidad de parámetros que estimar (pesos) y cuya entropía sea la menor posible.

Para lograr esto último, ponemos a la función de pérdida un término que restrinja los valores de los pesos a valores más pequeños.

Esto se denomina **weight regularization**. Equivale a agregar un término de costo a la función de pérdida por tener pesos cada vez más grandes.

Este costo viene en dos sabores:

- * L1: el costo es el proporcional al VALOR ABSOLUTO DE LOS COEFICIENTES DE LOS PESOS.
- * L2: el costo es proporcional al CUADRADO DEL VALOR DE LOS COEFICIENTES DE LOS PESOS.

Regularización – weight regularization.

¿cómo se hace en keras?

```
from keras import regularizers

model = models.Sequential()
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),
                        activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),
                        activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

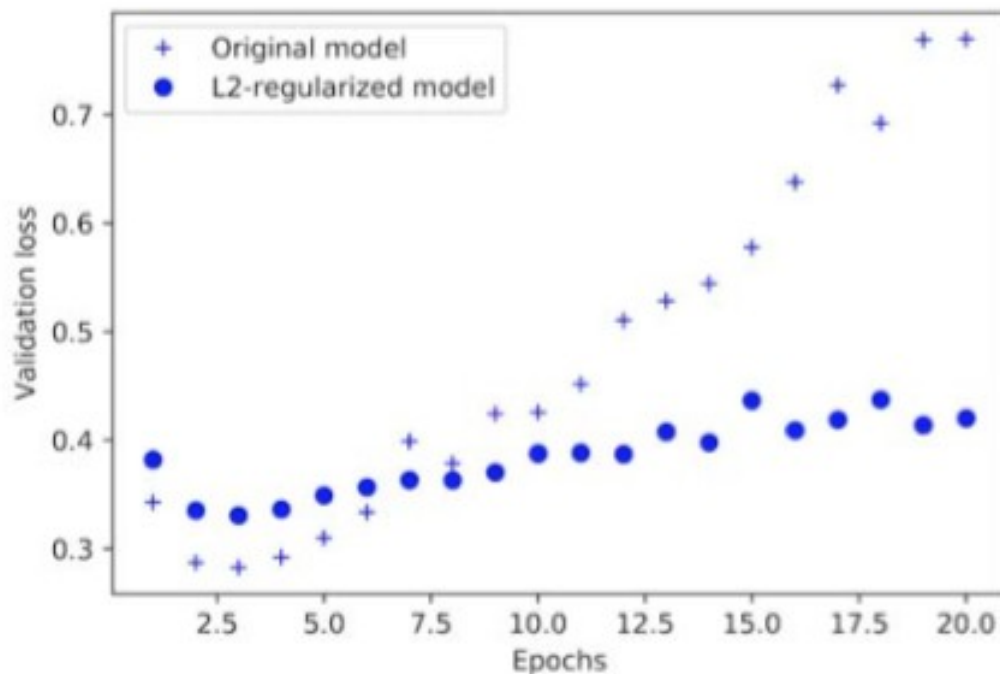
Red original con **regularización**:
10000 vector input
1era Capa escondida 16 nodos.
2da Capa escondida 16 nodos.
Neurona de salida con sigmoid.

El l2, significa que se añade la norma L2.

El número 0.001 significa que a cada coeficiente W_{ij} en la matriz de pesos sinápticos de la red se le sumará $0.001 * W_{ij}$.

Regularización – weight regularization.

¿cómo se hace en keras?



Vemos que la red con regularizador, tiende a tener un menor loss que sin, y además la tasa de subida del loss (overfitting) con regularización es realmente mucho menor que sin.

Regularización – dropout.

El dropout es la técnica más utilizada para regularización y posiblemente mas simple que el uso de de weight regularization.

Dropout se aplica a cada capa y consiste en setear a cero en forma aleatoria un número determinado de neuronas de la capa en la fase de entrenamiento.

En realidad, esto es un equivalente a “agregar ruido” a los valores de salida de las neuronas para prevenir el overfitting.

Regularización – dropout.

¿cómo se hace en Keras?

Red original con **dropout**:

10000 vector input

1era Capa escondida 16 nodos.

2da Capa escondida 16 nodos.

Neurona de salida con sigmoid.

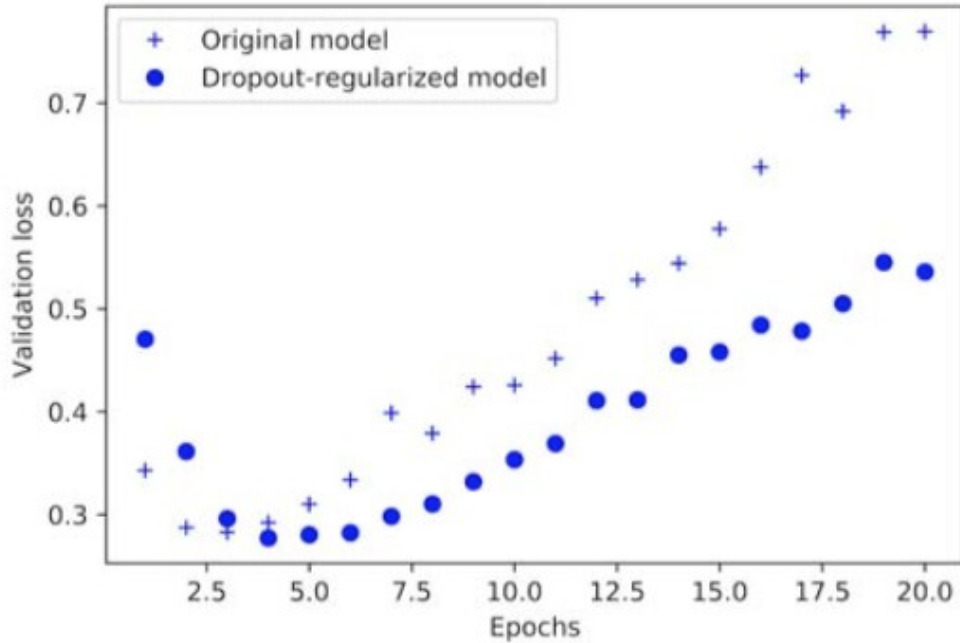
```
model = models.Sequential()  
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))  
model.add(layers.Dropout(0.5))  
model.add(layers.Dense(16, activation='relu'))  
model.add(layers.Dropout(0.5))  
model.add(layers.Dense(1, activation='sigmoid'))
```

Aquí 0.5 es el dropout rate, es decir, 50% de los valores se setean en cero.

Usualmente este valor está entre 0.2 y 0.5.

Regularización – dropout.

¿cómo se hace en Keras?



Vemos que la red con dropout tiende a tener un menor loss que sin.

Equivale a una reducción de capas escondidas (al comparar con ese tipo de regularización), pero sin sacrificar la capacidad de detección de patrones altamente no-lineales.